Experiment 13 — Case Study on Low-Code Limitations

Introduction

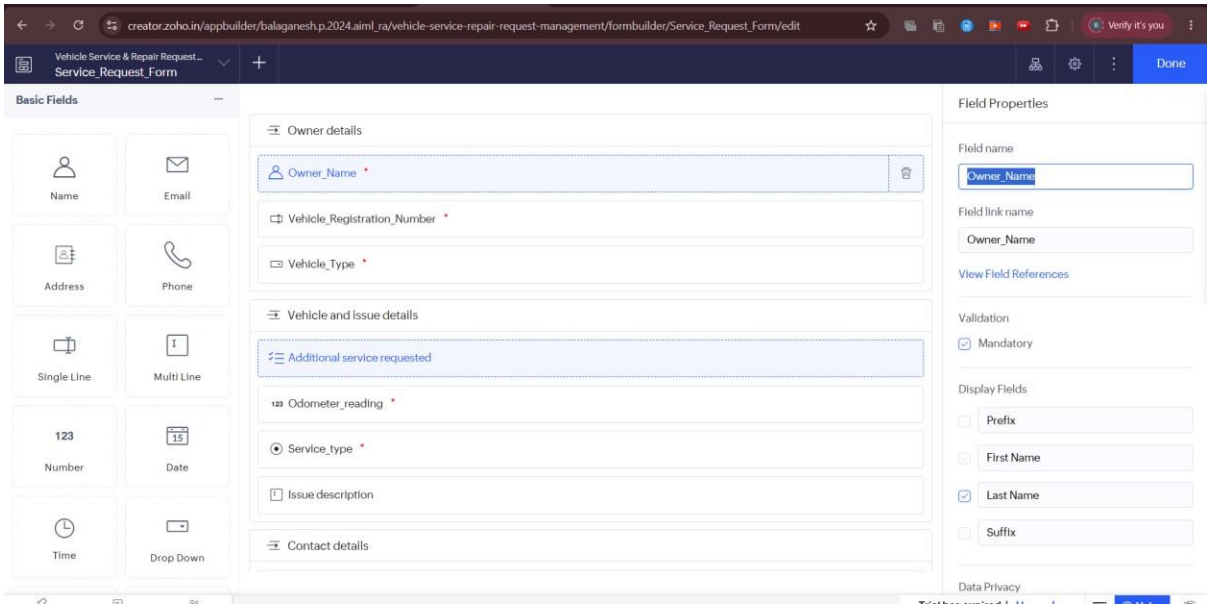
Low-code development is a software development approach that allows users to create applications using visual interfaces, drag-and-drop components, predefined templates, and minimal programming. Platforms such as Zoho Creator help users develop business applications quickly without writing large amounts of code. Low-code platforms are useful for automating workflows, managing data, creating reports, and developing web applications.

However, low-code development also has certain limitations. These limitations become more noticeable when applications require complex business logic, advanced customization, high performance, specialized integrations, or strict security controls.

1. Limited Customization

One of the major limitations of low-code platforms is the restriction on customization. Developers mainly use the components and features provided by the platform. Although some platforms support scripting and custom functions, they may not allow complete control over the application's internal structure.

For example, in a vehicle service management application developed using Zoho Creator, standard forms and reports can be created easily. However, implementing a highly customized vehicle tracking interface or a complex interactive service dashboard may require additional development or external tools

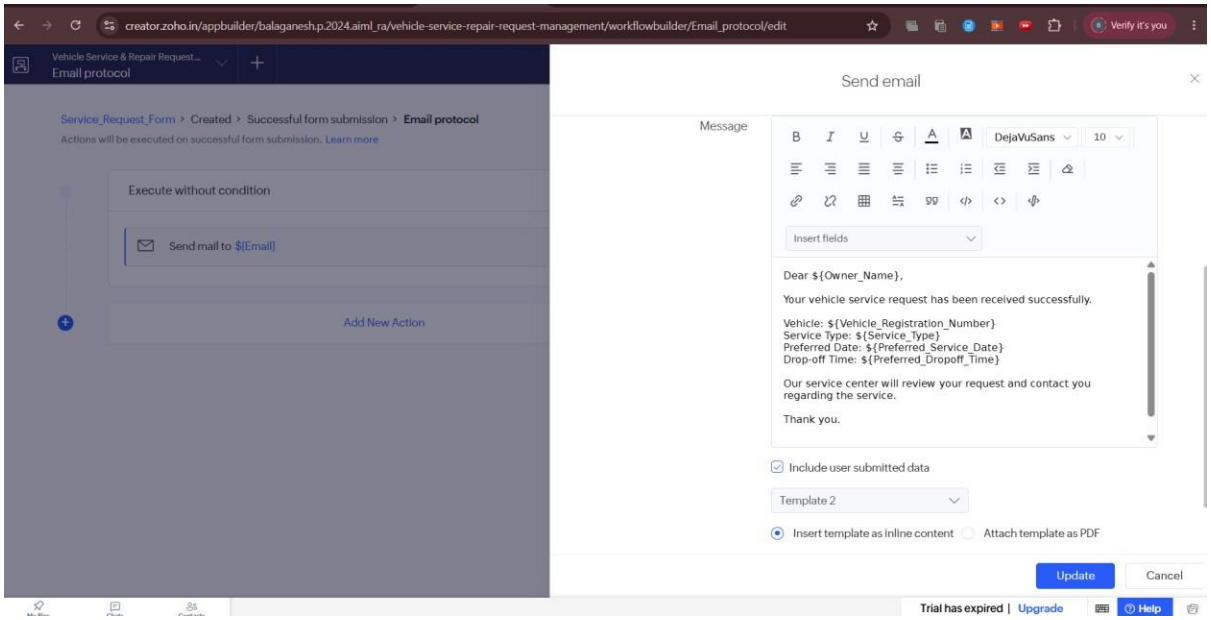


.

2. Complex Business Logic

Low-code platforms are suitable for simple workflows, but complex business processes can become difficult to implement. Advanced conditions, multiple dependent workflows, complicated calculations, and exception handling may require custom scripting.

For example, a vehicle service application may need to calculate repair costs, apply discounts, obtain approval for expensive repairs, update payment status, and send notifications. When several workflows depend on one another, debugging and maintaining the application can become difficult.



3. Performance and Scalability

As the number of records and users increases, application performance may decrease if forms, reports, workflows, or queries are not optimized. Large datasets and unnecessary processing can increase loading times and delay operations.

For example, a vehicle service centre with thousands of service requests may experience slower report generation if every report retrieves unnecessary fields or processes all records instead of using suitable filters.

Low-code applications may also have platform-specific limits on records, API calls, workflow execution, and other resources. Therefore, the application's expected workload must be considered during design.

4. Limited Integration with External Systems

Businesses often need to connect applications with payment gateways, accounting software, inventory systems, external databases, or third-party services. Although low-

code platforms provide integration options, some systems may not have ready-made connectors.

In such cases, developers may need APIs, custom functions, authentication configuration, or additional middleware. These integrations can require programming knowledge and may involve extra costs or technical restrictions.

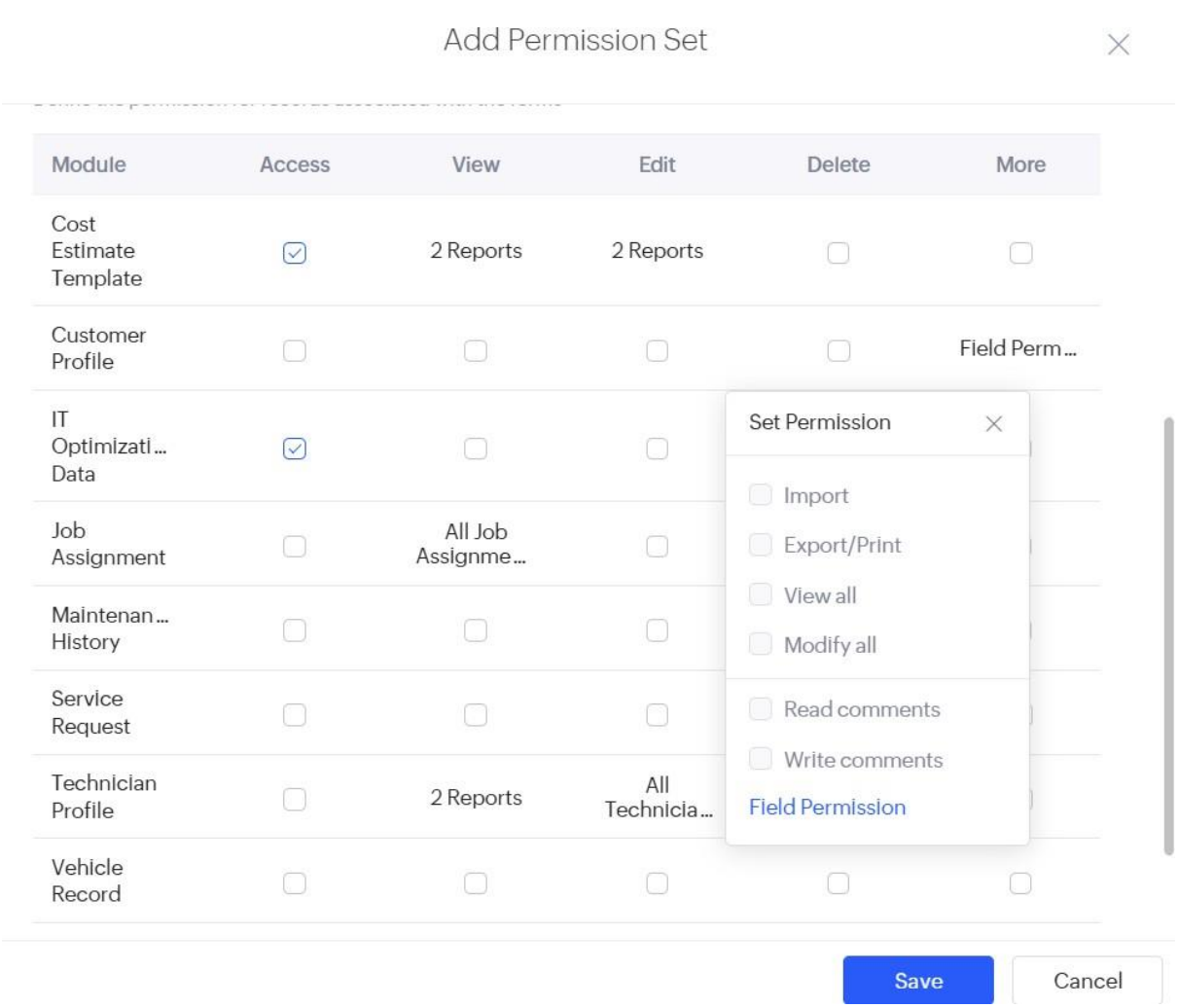
For example, connecting a vehicle service application to an external spare-parts inventory system may require custom API integration if a suitable built-in connector is unavailable.

5. Security and Data Privacy

Low-code platforms provide security features such as user permissions, access controls, and data-sharing settings. However, developers must configure these features correctly. Incorrect permissions can expose confidential information or allow unauthorized modifications.

For example, vehicle owners' phone numbers, email addresses, pickup addresses, and payment information should not be visible to unrelated users. Service staff should have access only to the records and operations necessary for their work.

Organizations must also consider data privacy requirements, authentication, audit trails, backups, and the platform's data storage policies. Security cannot be guaranteed simply by choosing a low-code platform.



6. Vendor Lock-In

Vendor lock-in occurs when an application becomes highly dependent on a particular platform. Forms, workflows, database structures, and scripts may use platform-specific features that cannot be transferred directly to another development environment.

If an organization decides to migrate its application to a different platform, some components may need to be redesigned or rewritten. Data export alone does not necessarily transfer the application's complete functionality.

For example, moving a Zoho Creator vehicle service application to a custom application developed using Python, Java, or another framework may require rebuilding its forms, workflows, permissions, and reports.

7. Limited Control over Source Code

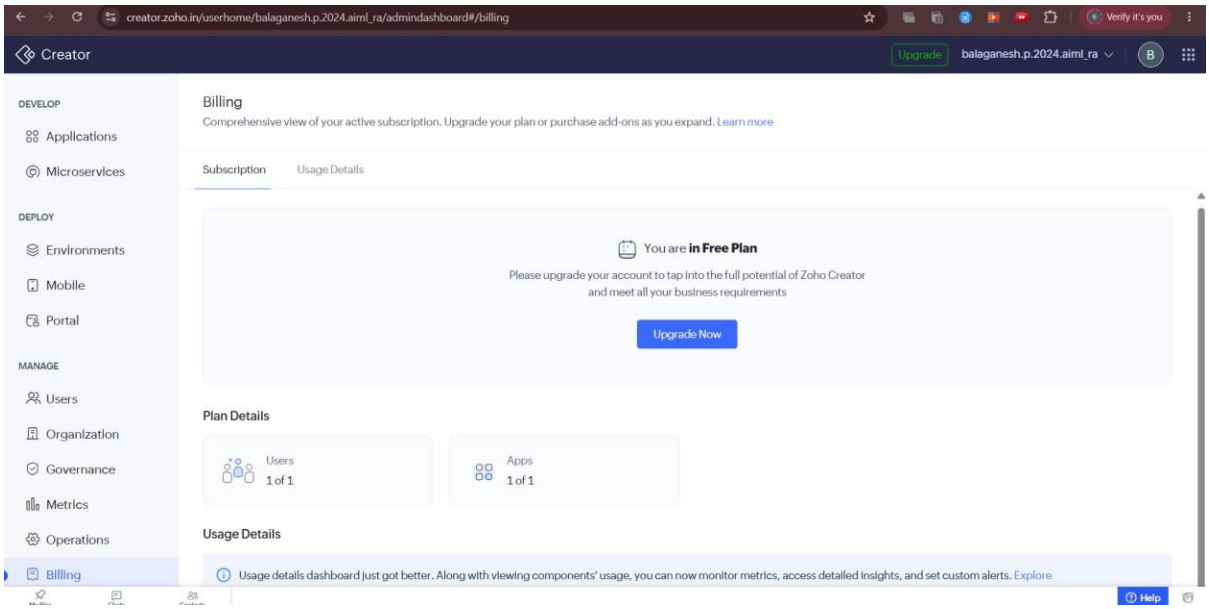
Traditional software development gives developers greater control over source code, architecture, libraries, and deployment. Low-code platforms abstract many of these details to simplify development.

This abstraction can limit the ability to implement specialized algorithms, use unsupported libraries, optimize internal processing, or modify the platform's underlying behavior. Advanced requirements may therefore need pro-code extensions or external services.

8. Platform Dependency and Pricing

Low-code applications depend on the platform's availability, features, pricing structure, and service policies. Changes in subscription plans, usage limits, supported features, or licensing can affect the application's operating cost.

Some advanced capabilities may only be available in particular plans. Organizations must evaluate user limits, storage, integrations, automation quotas, and future costs before selecting a platform.



9. Testing and Maintenance Challenges

Although low-code development reduces the amount of code written, it does not eliminate the need for testing and maintenance. Developers must verify form validation, calculations, workflow execution, access permissions, integrations, and error handling.

When many workflows interact, a small change may affect other parts of the application. Proper documentation, version management, backups, and regression testing are necessary to keep the application reliable.

For example, changing a service-status workflow may unintentionally affect payment processing or customer notifications if the dependencies are not tested carefully.

10. Suitability for Specialized Applications

Low-code platforms are not ideal for every type of software. Applications requiring intensive real-time processing, advanced graphics, specialized hardware access, highly optimized algorithms, or complete control over infrastructure may be better suited to traditional programming or a hybrid approach.

A vehicle service management system is well suited to low-code development because it mainly involves forms, records, reports, notifications, and approval workflows. However, advanced live vehicle tracking, complex predictive maintenance algorithms, or specialized integrations may require external services or custom code.

Conclusion

Low-code development offers faster application creation, reduced coding effort, and easier workflow automation. Nevertheless, it has limitations involving customization, complex business logic, performance, integration, security, portability, source-code control, cost, and maintenance.

In the Zoho Creator vehicle service management project, low-code tools are effective for handling service requests, vehicle details, repair progress, costs, and customer notifications. More advanced requirements may need custom scripting, external integrations, or pro-code extensions.

Therefore, low-code platforms should be selected according to the application's requirements, expected scale, security needs, budget, and future development plans. A combination of low-code and traditional programming can provide flexibility while retaining the advantages of rapid application development.